VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT
on

Machine Learning (23CS6PCMAL)

Submitted by

Gopika Pushparajan (1BM23CS101)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Jan-2026 to May-2026

B.M.S. College of Engineering,

Bull Temple Road, Bangalore 560019

(Affiliated To Visvesvaraya Technological University, Belgaum)

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "Machine Learning (23CS6PCMAL)" carried out by **Gopika Pushparajan (1BM23CS101),** who is a bonafide student of **B.M.S. College of Engineering.** It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Machine Learning (23CS6PCMAL) work prescribed for the said degree.

D. Chaithra Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE

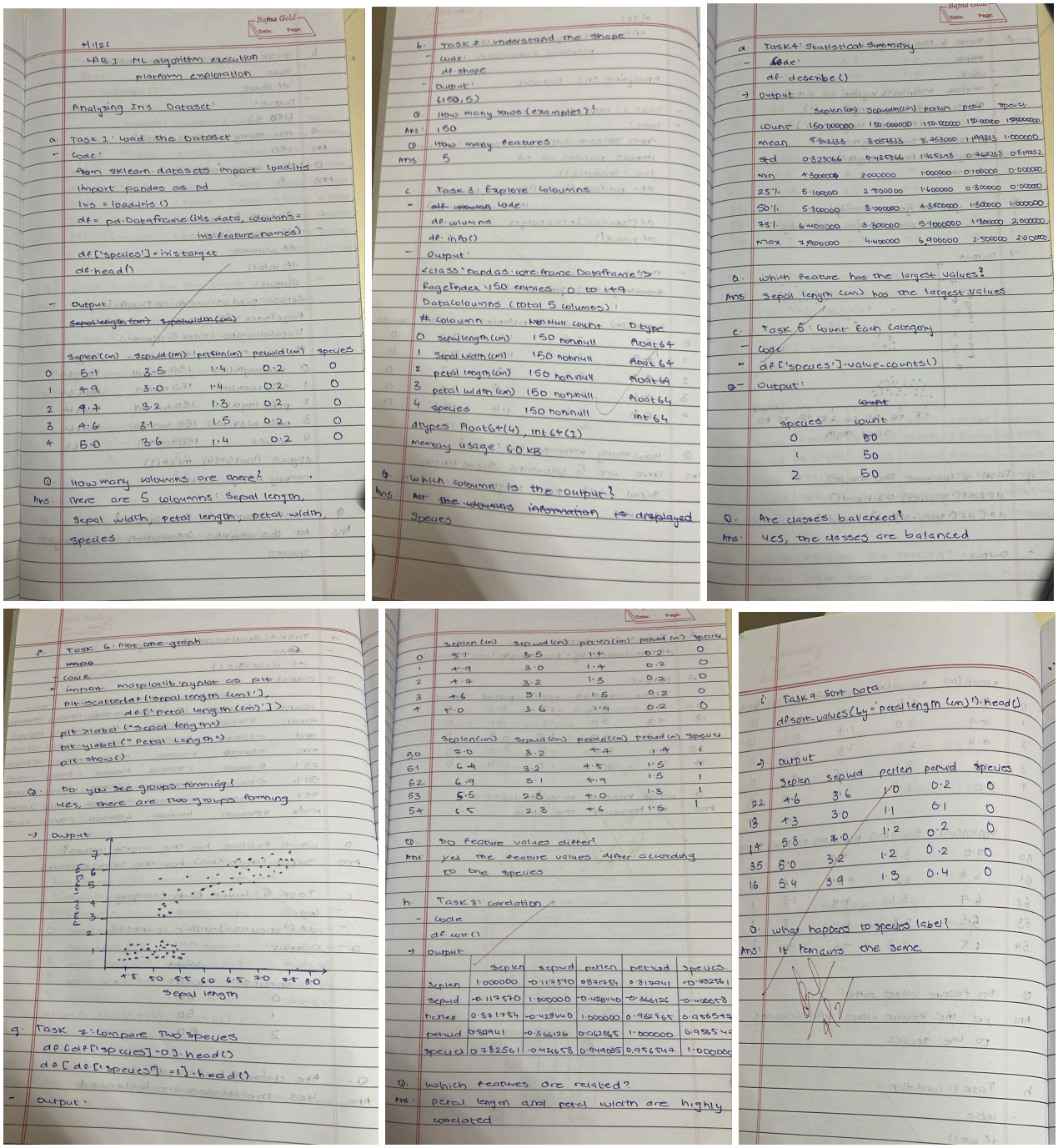
Index

Github Link:
https://github.com/gopikapushparajan/ML-LAB

Program 1

Write a python program to import and export data using Pandas library functions

Screenshot



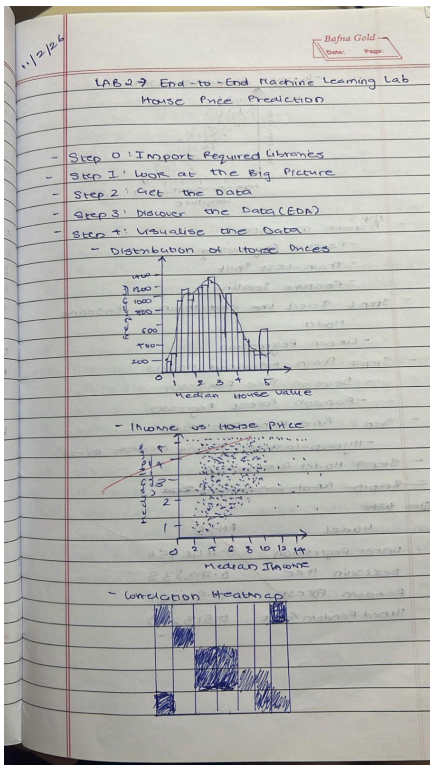
Code:

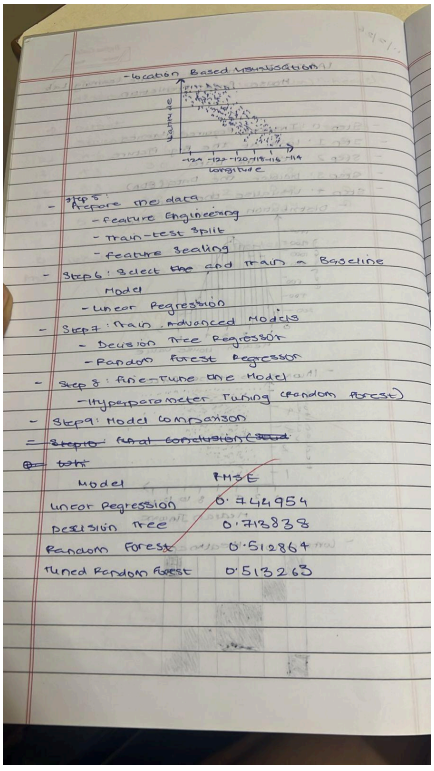
```
from sklearn.datasets import load_iris
import pandas as pd
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df[‘species’] = iris.target
df.head()
df.columns
df.info()
df.describe()
df[‘species’].value_counts()

import matplotlib.pyplot as plt

plt.scatter(df['sepal length (cm)'], df['petal length (cm)'])

plt.xlabel("Sepal length")

plt.ylabel("Petal length")

plt.show()

df[df['species']==0].head()

df[df['species']==1].head()

df.corr()

df.sort_values(by='petal length (cm)').head()
```

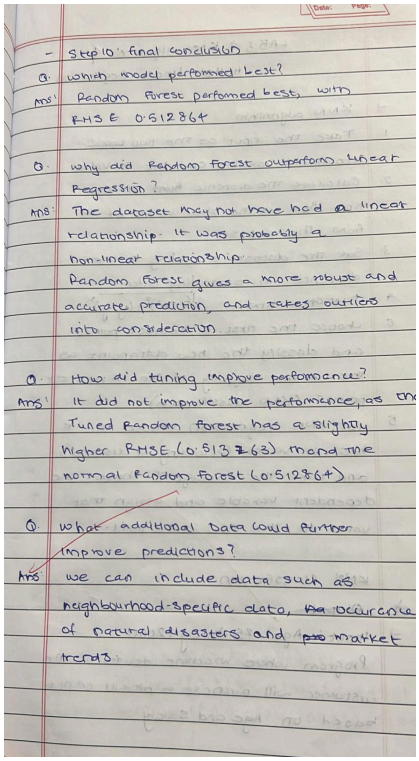
Program 2

Demonstrate various data pre-processing techniques for a given dataset

Screenshot







Code:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.datasets import fetch_california_housing

from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LinearRegression
```

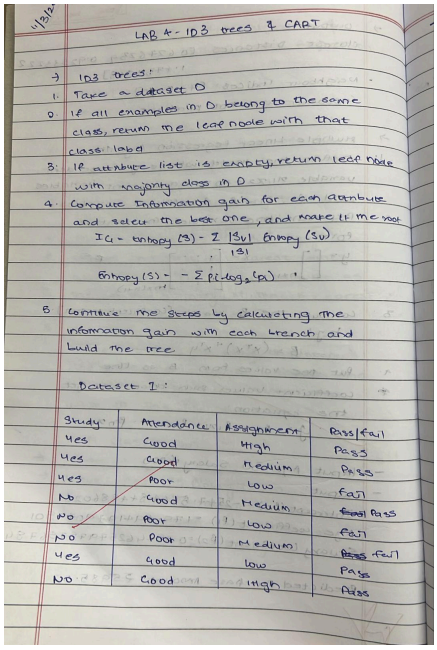
```
from sklearn.tree import DecisionTreeRegressor

from sklearn.ensemble import RandomForestRegressor

from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

housing = fetch_california_housing(as_frame=True)

df = housing.frame

df.head()

df.shape

df.info()

plt.figure()

sns.histplot(df['MedHouseVal'], bins=30, kde=True)

plt.xlabel('Median House Value')

plt.ylabel('Frequency')

plt.title('Distribution of House Prices')

plt.show()

plt.figure()

sns.scatterplot(x=df['MedInc'], y=df['MedHouseVal'], alpha=0.5)

plt.xlabel('Median Income')

plt.ylabel('Median House Value')

plt.title('Income vs House Price')

plt.show()

plt.figure(figsize=(10,8))
```

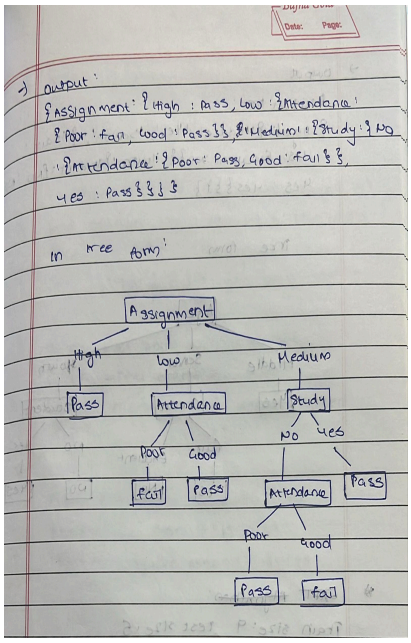
```
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')

plt.title('Feature Correlation Heatmap')

plt.show()

plt.figure()

plt.scatter(df['Longitude'], df['Latitude'],

c=df['MedHouseVal'], cmap='viridis', s=5)

plt.colorbar(label='House Value')

plt.xlabel('Longitude')

plt.ylabel('Latitude')

plt.title('House Prices by Location')

plt.show()
```

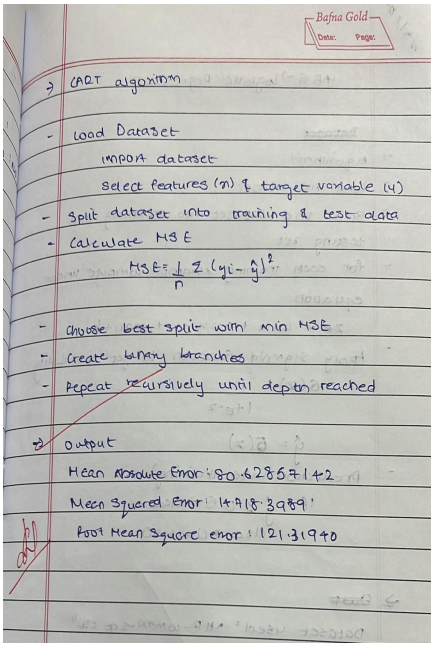
Program 3

Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.

Screenshot:







Code:

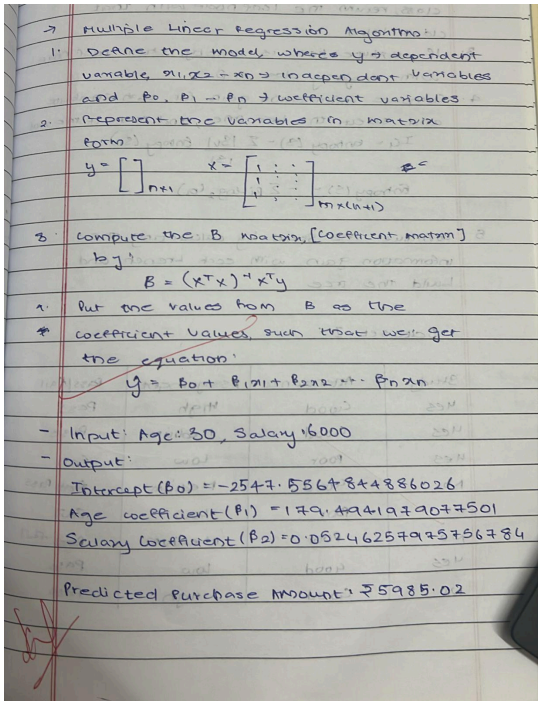
```
import pandas as pd

from google.colab import files

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import accuracy_score, confusion_matrix, precision_score, recall_score,
f1_score

from sklearn.preprocessing import LabelEncoder

uploaded = files.upload()

file_path = list(uploaded.keys())[0]

df = pd.read_csv(file_path)

print("\nDataset Preview:\n", df.head())
```

```
target_column = input("\nEnter the target column name: ")
df = df.dropna()
le = LabelEncoder()
for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = le.fit_transform(df[col])
X = df.drop(columns=[target_column])
y = df[target_column]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted', zero_division=0)
recall = recall_score(y_test, y_pred, average='weighted', zero_division=0)
f1 = f1_score(y_test, y_pred, average='weighted', zero_division=0)
print("Accuracy:", accuracy)
print("\nConfusion Matrix:\n", conf_matrix)
print("\nPrecision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)
```

Program 4

Implement Linear and Multi-Linear Regression algorithm using appropriate dataset

Screenshot:



Code:

```
import pandas as pd

from google.colab import files

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

import numpy as np

import matplotlib.pyplot as plt

uploaded = files.upload()

file_path = list(uploaded.keys())[0]

df = pd.read_csv(file_path)
```

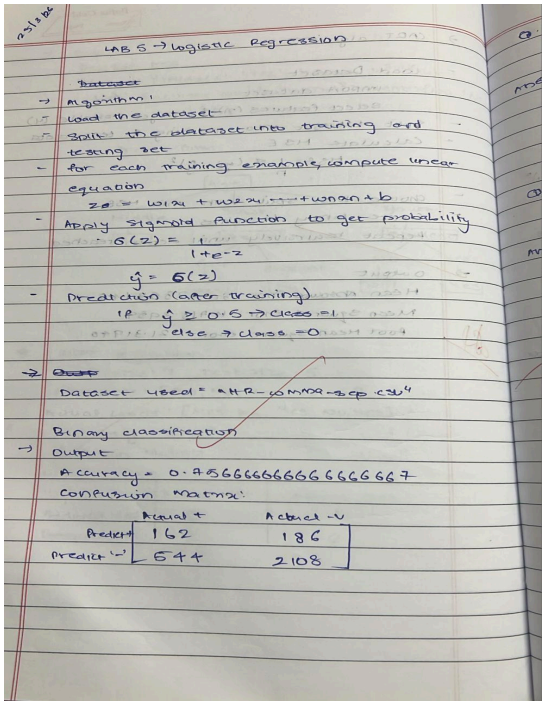
```
print("\nColumns:\n", df.columns)
print("\nPreview:\n", df.head())
feature_column = input("\nEnter ONE feature column (numeric): ")
target_column = input("Enter the target column (numeric): ")
df = df[[feature_column, target_column]].dropna()
X = df[[feature_column]]
y = df[target_column]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
print("\n--- Simple Linear Regression Evaluation ---")
print("MAE:", mae)
print("MSE:", mse)
print("RMSE:", rmse)
print("R2 Score:", r2)
```

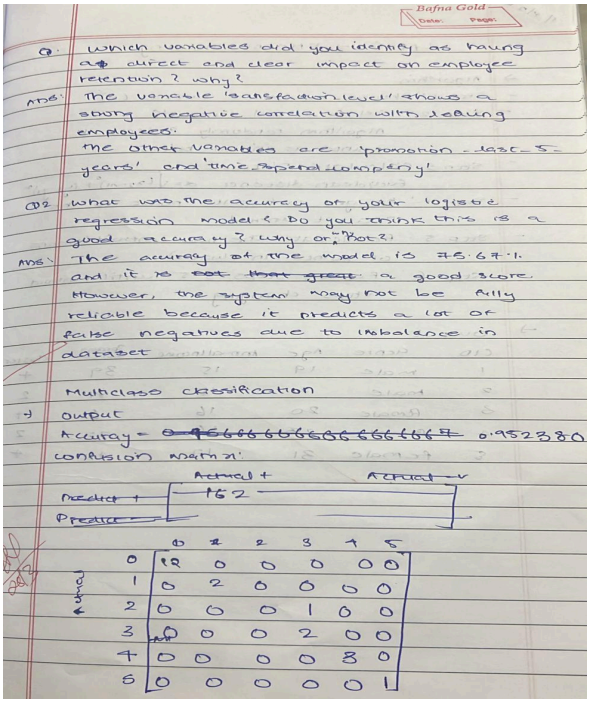
```
plt.figure()

plt.scatter(X_test, y_test, label="Actual Data")

plt.plot(X_test, y_pred, label="Regression Line")

plt.xlabel(feature_column)

plt.ylabel(target_column)

plt.title("Simple Linear Regression")

plt.legend()

plt.show()
```

Program 5

Build Logistic Regression Model for a given dataset

Screenshot:





Code:

```
import pandas as pd

from google.colab import files

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, confusion_matrix, precision_score, recall_score, f1_score

from sklearn.preprocessing import LabelEncoder, StandardScaler

uploaded = files.upload()

file_path = list(uploaded.keys())[0]

df = pd.read_csv(file_path)
```

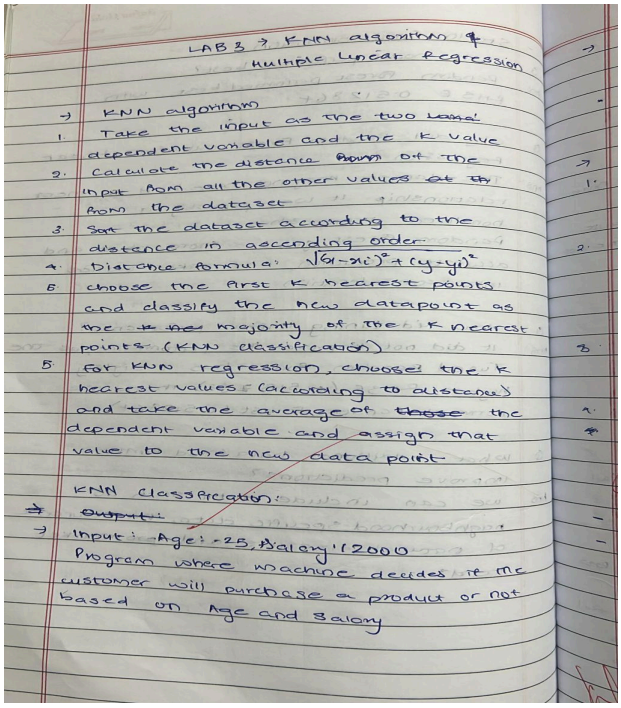
```
print("\nDataset Preview:\n", df.head())
target_column = input("\nEnter the target column name")
df = df.dropna()
le = LabelEncoder()
for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = le.fit_transform(df[col])
X = df.drop(columns=[target_column])
y = df[target_column]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted', zero_division=0)
recall = recall_score(y_test, y_pred, average='weighted', zero_division=0)
f1 = f1_score(y_test, y_pred, average='weighted', zero_division=0)
```

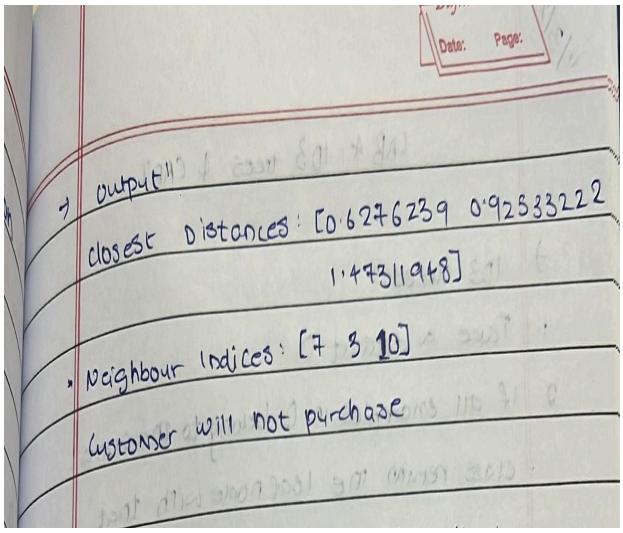
```
print("\n--- Logistic Regression Evaluation ---")

print("Accuracy:", accuracy)

print("\nConfusion Matrix:\n", conf_matrix)

print("\nPrecision:", precision)

print("Recall:", recall)

print("F1 Score:", f1)
```

Program 6

Build KNN Classification model for a given dataset.

Screenshot:





Code:

```
import pandas as pd

from google.colab import files

from sklearn.model_selection import train_test_split

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score, confusion_matrix, precision_score, recall_score, f1_score

from sklearn.preprocessing import LabelEncoder, StandardScaler

uploaded = files.upload()

file_path = list(uploaded.keys())[0]

df = pd.read_csv(file_path)
```

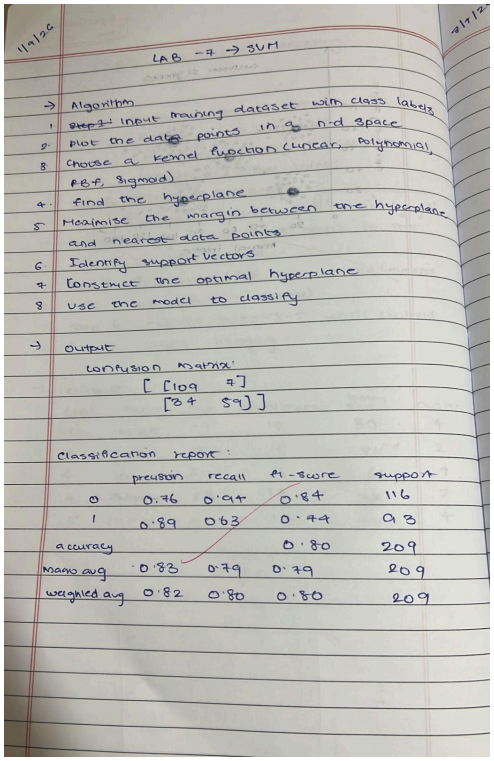
```
print("\nDataset Preview:\n", df.head())
target_column = input("\nEnter the target column name: ")
df = df.dropna()
le = LabelEncoder()
for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = le.fit_transform(df[col])
X = df.drop(columns=[target_column])
y = df[target_column]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted', zero_division=0)
recall = recall_score(y_test, y_pred, average='weighted', zero_division=0)
f1 = f1_score(y_test, y_pred, average='weighted', zero_division=0)
```

```
print("\n--- KNN Evaluation ---")

print("Accuracy:", accuracy)

print("\nConfusion Matrix:\n", conf_matrix)

print("\nPrecision:", precision)

print("Recall:", recall)

print("F1 Score:", f1)
```

Program 7

Build Support vector machine model for a given dataset

Screenshot:



LAB - 7 → SVM

→ Algorithm

1. ~~Step1:~~ Input training dataset with class labels
2. Plot the data points in n-d space
3. Choose a kernel function (Linear, Polynomial, RBF, Sigmoid)
4. Find the hyperplane
5. Maximise the margin between the hyperplane and nearest data points
6. Identify support vectors
7. Construct the optimal hyperplane
8. Use the model to classify

→ output

Confusion Matrix:

[[109 7]
[34 59]]

Classification report:

	precision	recall	f1-score	support
0	0.76	0.94	0.84	116
1	0.89	063	0.74	93
accuracy			0.80	209
macro avg	0.83	0.79	0.79	209
weighted avg	0.82	0.80	0.80	209

Code:

```
import pandas as pd

from google.colab import files

from sklearn.model_selection import train_test_split

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score, confusion_matrix, precision_score, recall_score, f1_score

from sklearn.preprocessing import LabelEncoder, StandardScaler

file_path = list(uploaded.keys())[0]

df = pd.read_csv(file_path)
```

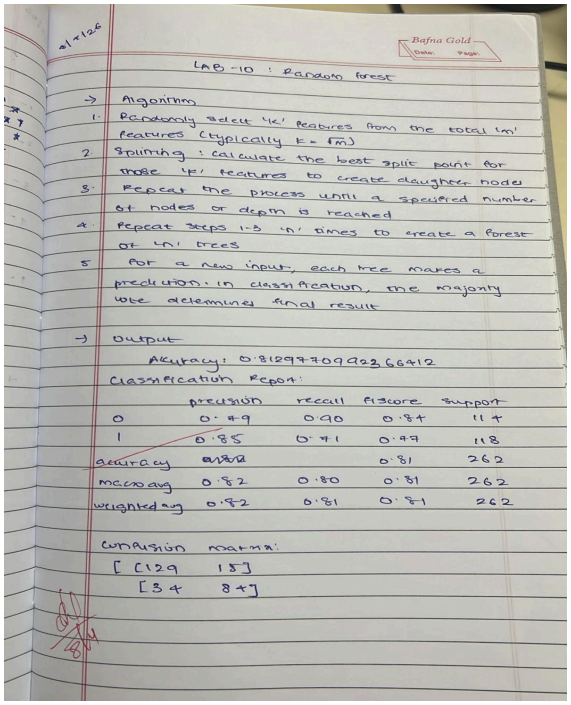
```
print("\nDataset Preview:\n", df.head())
target_column = input("\nEnter the target column name: ")
df = df.dropna()
le = LabelEncoder()
for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = le.fit_transform(df[col])
X = df.drop(columns=[target_column])
y = df[target_column]
scaler = StandardScaler()
X = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = SVC(kernel='rbf')   # you can try 'linear', 'poly'
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted', zero_division=0)
recall = recall_score(y_test, y_pred, average='weighted', zero_division=0)
f1 = f1_score(y_test, y_pred, average='weighted', zero_division=0)
```

```
print("\n--- SVM Evaluation ---")

print("Accuracy:", accuracy)

print("\nConfusion Matrix:\n", conf_matrix)

print("\nPrecision:", precision)

print("Recall:", recall)

print("F1 Score:", f1)
```

Program 8

Implement Random forest ensemble method on a given dataset

Screenshot:



Code:

```
import pandas as pd

from google.colab import files

from sklearn.model_selection import train_test_split

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score, confusion_matrix, precision_score, recall_score, f1_score

from sklearn.preprocessing import LabelEncoder

uploaded = files.upload()

file_path = list(uploaded.keys())[0]

df = pd.read_csv(file_path)
```

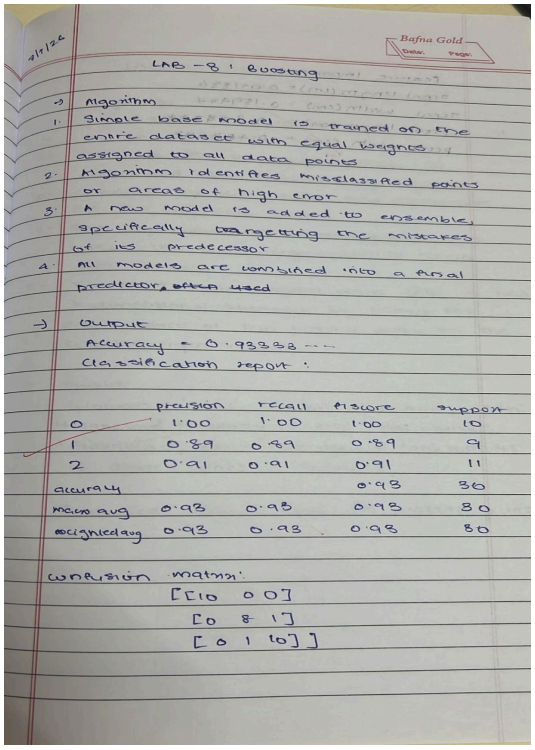
```
print("\nDataset Preview:\n", df.head())
target_column = input("\nEnter the target column name: ")
df = df.dropna()
le = LabelEncoder()
for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = le.fit_transform(df[col])
X = df.drop(columns=[target_column])
y = df[target_column]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = RandomForestClassifier(
    n_estimators=100,     # number of trees
    max_depth=None,
    random_state=42
)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted', zero_division=0)
recall = recall_score(y_test, y_pred, average='weighted', zero_division=0)
```

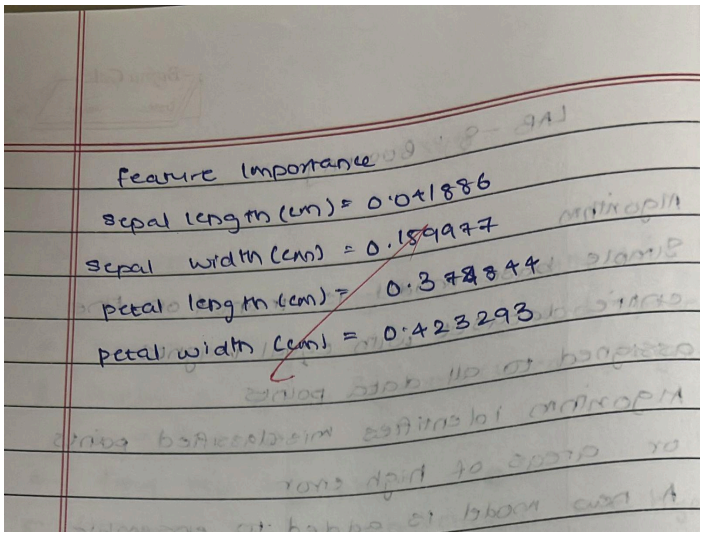
```
f1 = f1_score(y_test, y_pred, average='weighted', zero_division=0)

print("\n--- Random Forest Evaluation ---")

print("Accuracy:", accuracy)

print("\nConfusion Matrix:\n", conf_matrix)

print("\nPrecision:", precision)

print("Recall:", recall)

print("F1 Score:", f1)
```

Program 9

Implement Boosting ensemble method on a given dataset.

Screenshot:





Code:

```
import pandas as pd

from google.colab import files

from sklearn.model_selection import train_test_split

from sklearn.ensemble import AdaBoostClassifier

from sklearn.metrics import accuracy_score, confusion_matrix, precision_score, recall_score,
f1_score

from sklearn.preprocessing import LabelEncoder

uploaded = files.upload()

file_path = list(uploaded.keys())[0]

df = pd.read_csv(file_path)
```

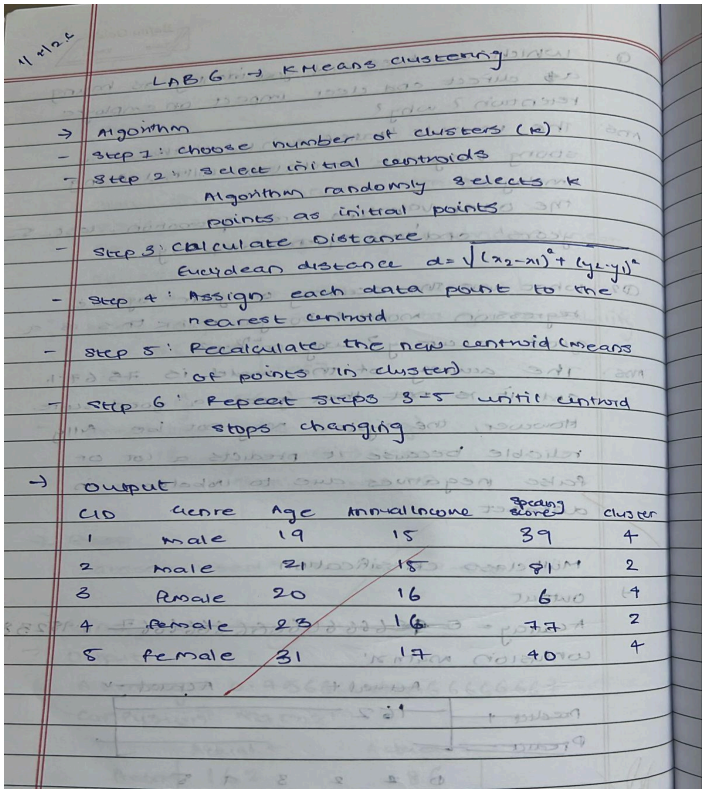
```
print("\nDataset Preview:\n", df.head())
target_column = input("\nEnter the target column name: ")
df = df.dropna()
le = LabelEncoder()
for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = le.fit_transform(df[col])
X = df.drop(columns=[target_column])
y = df[target_column]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = AdaBoostClassifier(
    n_estimators=50,
    learning_rate=1.0,
    random_state=42
)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)

precision = precision_score(y_test, y_pred, average='weighted', zero_division=0)
```

```
recall = recall_score(y_test, y_pred, average='weighted', zero_division=0)

f1 = f1_score(y_test, y_pred, average='weighted', zero_division=0)

print("\n--- AdaBoost Evaluation ---")

print("Accuracy:", accuracy)

print("\nConfusion Matrix:\n", conf_matrix)

print("\nPrecision:", precision)

print("Recall:", recall)

print("F1 Score:", f1)
```

Program 10

Build k-Means algorithm to cluster a set of data stored in a .CSV file.

Screenshot:



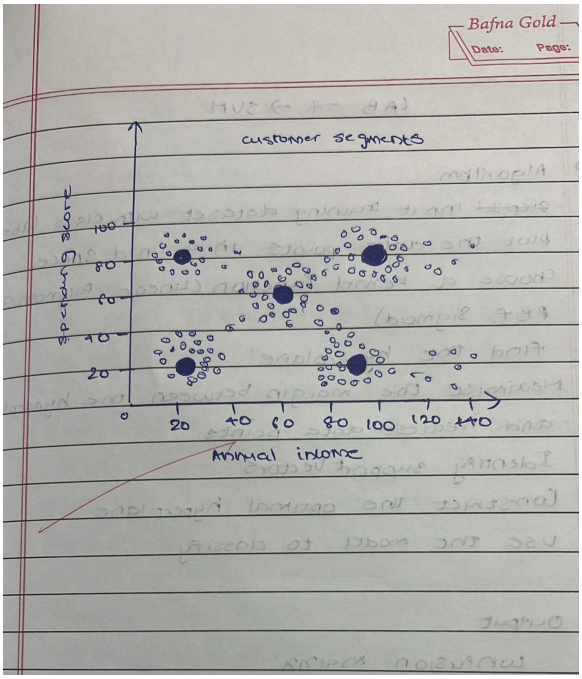
LAB 6 → KMeans clustering

→ Algorithm

- Step 1: choose number of clusters (K).
- Step 2: select initial centroids
  Algorithm randomly selects K points as initial points
- Step 3: Calculate distance
  Euclidean distance $d=\sqrt{(x_2-x_1)^2+(y_2-y_1)^2}$
- Step 4: Assign each data point to the nearest centroid
- Step 5: Recalculate the new centroid (means of points in cluster)
- Step 6: Repeat steps 3-5 until centroid stops changing

→ Output

CID	Genre	Age	Annual Income	Spending Score	cluster
1	Male	19	15	39	4
2	Male	21	15	81	2
3	Female	20	16	6	4
4	Female	23	16	77	2
5	Female	31	17	40	4



Code:

```
import pandas as pd

from google.colab import files

from sklearn.cluster import KMeans

from sklearn.preprocessing import StandardScaler

import matplotlib.pyplot as plt

uploaded = files.upload()

file_path = list(uploaded.keys())[0]

df = pd.read_csv(file_path)
```

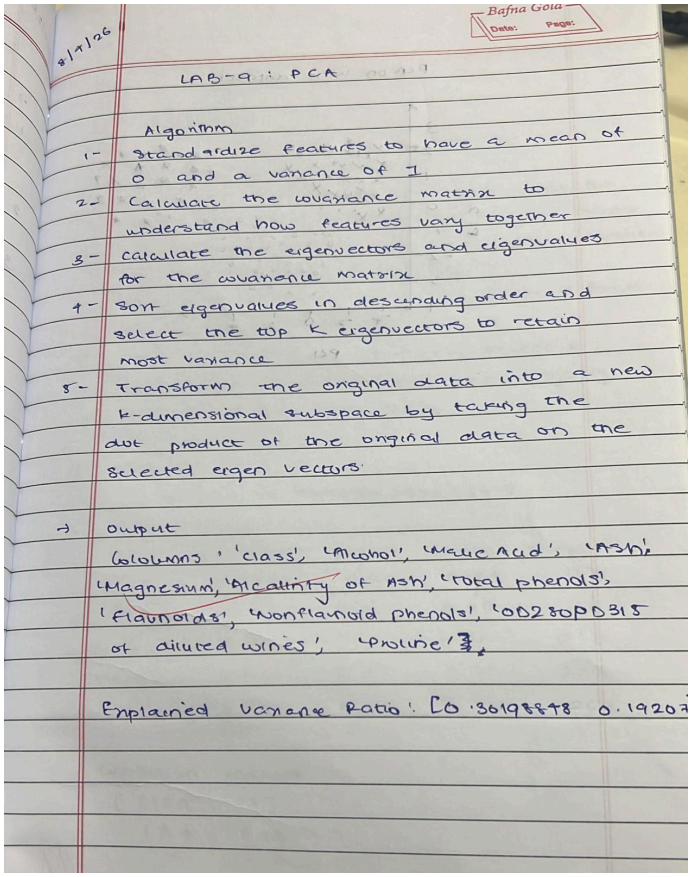
```
print("\nColumns in dataset:\n", df.columns)
print("\nDataset Preview:\n", df.head())
col1 = input("\nEnter first feature column name: ")
col2 = input("Enter second feature column name: ")
X = df[[col1, col2]].dropna()
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
k = int(input("\nEnter number of clusters (k): "))
kmeans = KMeans(n_clusters=k, random_state=42)
y_kmeans = kmeans.fit_predict(X_scaled)
plt.figure()
for i in range(k):
    plt.scatter(
        X.iloc[y_kmeans == i, 0],
        X.iloc[y_kmeans == i, 1],
        label=f'Cluster {i}'
    )
centroids = scaler.inverse_transform(kmeans.cluster_centers_)
plt.scatter(
    centroids[:, 0],
    centroids[:, 1],
    s=200,
    c='black',
    label='Centroids'
```

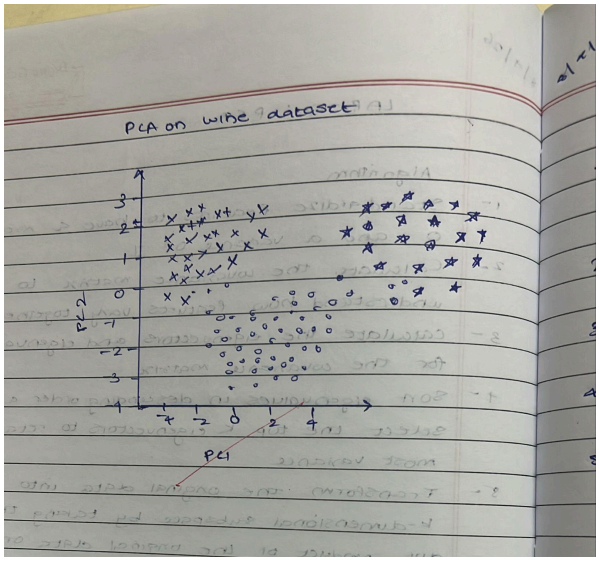
```
)
plt.xlabel(col1)
plt.ylabel(col2)
plt.title('K-Means Clustering')
plt.legend()
plt.show()
```

Program 11

Implement Dimensionality reduction using Principle Component Analysis (PCA) method

Screenshot:





Code:

```
import pandas as pd
from google.colab import files
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
uploaded = files.upload()
file_path = list(uploaded.keys())[0]
```

```
df = pd.read_csv(file_path)
print("\nColumns:\n", df.columns)
print("\nPreview:\n", df.head())
use_target = input("\nDo you want to use a target column for coloring? (yes/no): ")
if use_target.lower() == 'yes':
    target_column = input("Enter target column name: ")
    y = df[target_column]
    X = df.drop(columns=[target_column])
else:
    y = None
    X = df.copy()
X = X.dropna()
X = pd.get_dummies(X, drop_first=True)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
print("\nExplained Variance Ratio:", pca.explained_variance_ratio_)
plt.figure()
if y is not None:
    plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y)
else:
    plt.scatter(X_pca[:, 0], X_pca[:, 1])
```

```
plt.xlabel("Principal Component 1")

plt.ylabel("Principal Component 2")

plt.title("PCA Visualization")

plt.show()
```